

LAB 2 – Introduction to Natural Language Processing and Subword Tokenization

Project Goal

Create a lightweight, package-free Natural Language Processing (NLP) pipeline in Python, implementing sentence segmentation, word tokenization, noise filtering, and a custom Byte Pair Encoding (BPE) subword vocabulary merge loop, with detailed analysis of underlying computational mechanics.

Architecture Overview

This lab demonstrates the primary cleaning and slicing operations required to convert raw text into token structures consumable by large language models. Students write a Python script using standard libraries to split sentences and words, remove grammatical noise, and execute character-merging loops to learn how subword tokenizers build vocabularies.

Technologies Used

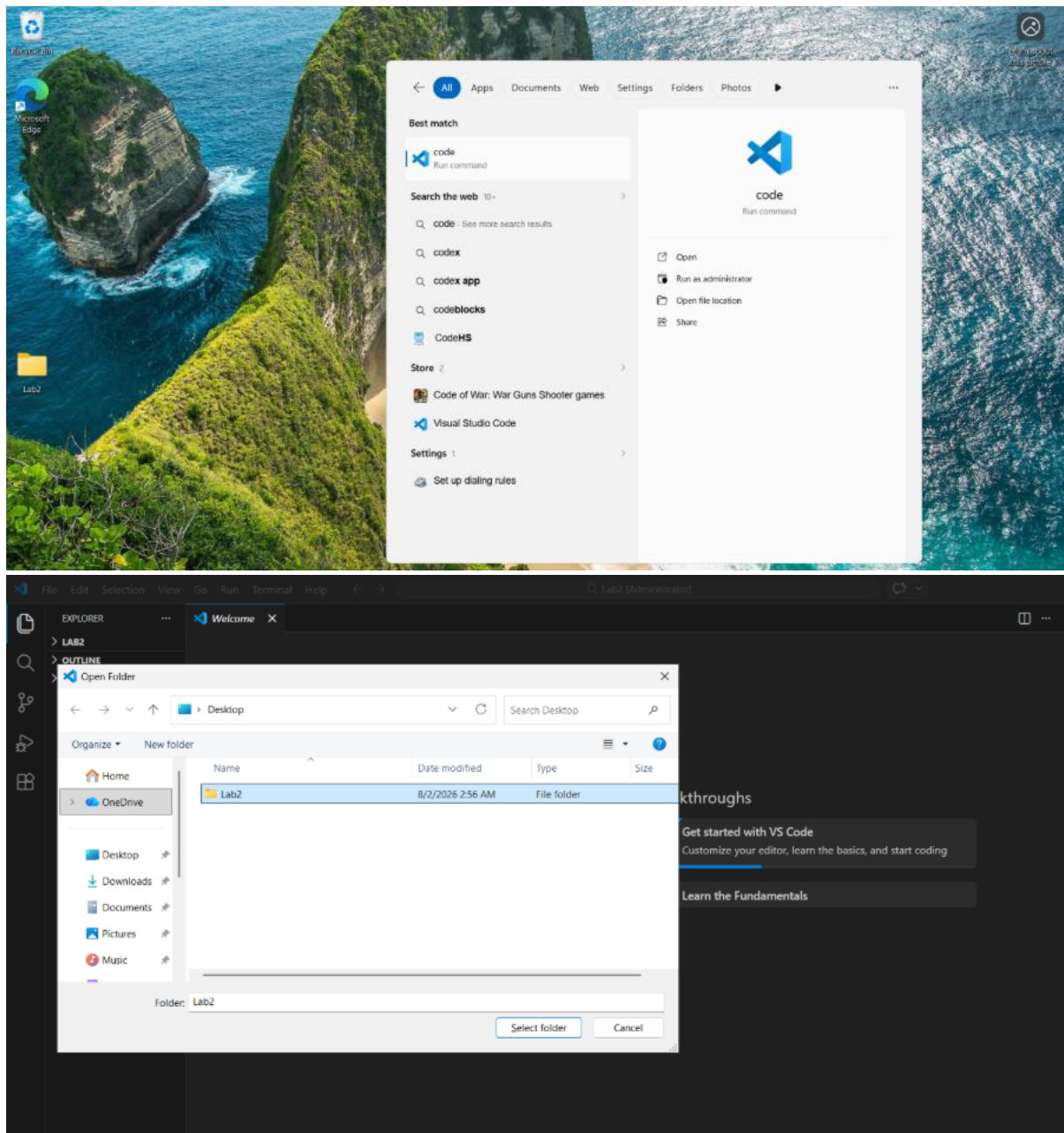
- Python 3.10+
- VS Code
- re (Regular Expressions)
- collections

Lab Implementation

Step 1 — Create the Lab Folder and Open VS Code

Execution

- Right-click on the Desktop and select New → Folder.
- Name the folder Lab2.
- Click the Windows Start button.
- Type Code in the Windows search bar.
- Click the Code application from the search results.
- In VS Code, select File → Open Folder and open the Lab2 folder. Alternatively, press Ctrl + K, then Ctrl + O, select the Lab2 folder, and click Select Folder.



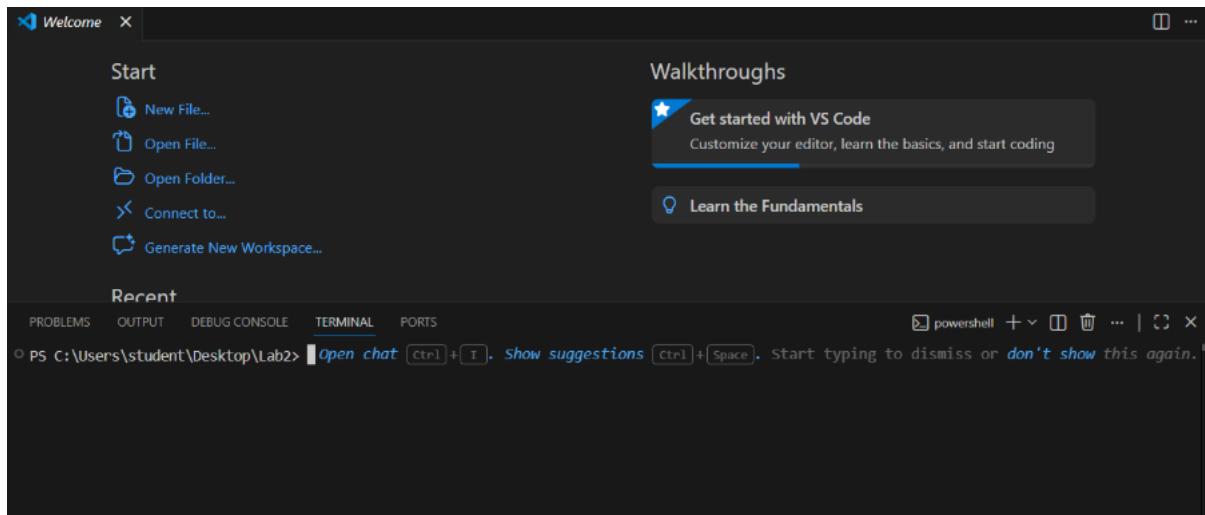
Expected Output / Outcome

The Lab2 folder is opened in Visual Studio Code and is displayed in the Explorer pane on the left. The title bar at the top of the window also updates to show "Lab2" as the open workspace, confirming VS Code is pointed at the right folder.

Step 2 — Open Integrated Terminal

Execution

- From the top menu, click Terminal → New Terminal.
- Or press Ctrl + Shift + `.



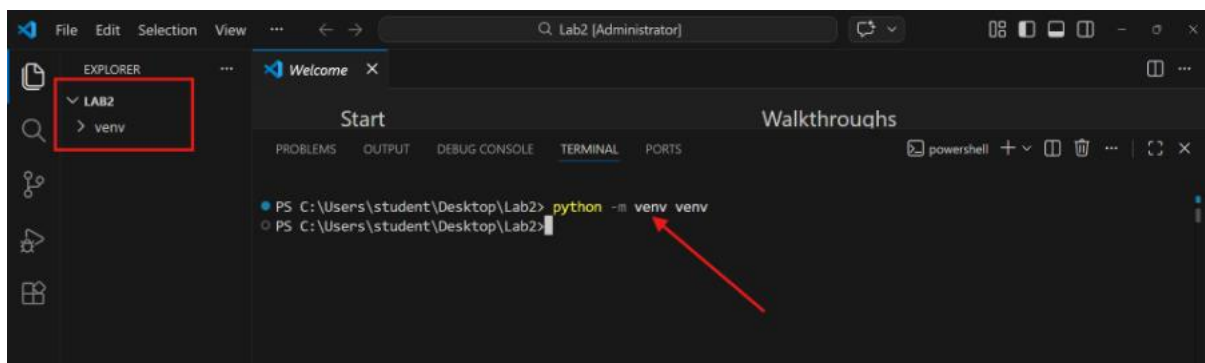
Expected Output / Outcome

An integrated terminal opens at the bottom of the VS Code window and is ready to accept commands. The prompt shows the current path ending in \Lab2, confirming the terminal is anchored to the correct working directory before any commands are run.

Step 3 — Initialize Python Virtual Environment

Execution

```
python -m venv venv
```



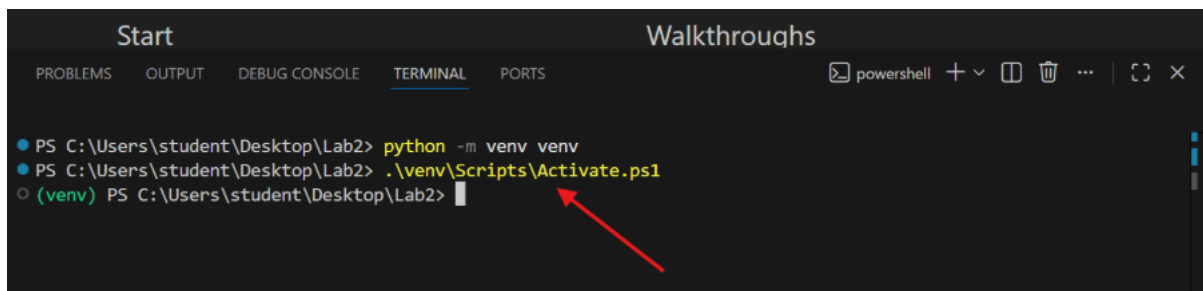
Expected Output / Outcome

A virtual environment folder named venv is created inside the Lab2 directory. Opening it in the Explorer pane shows subfolders such as Scripts (or bin on macOS/Linux), Lib, and a pyvenv.cfg file, all confirming a clean, isolated Python environment separate from the system installation.

Step 4 — Activate the Virtual Environment

Execution

```
.\venv\Scripts\Activate.ps1
```



```
Start Walkthroughs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\student\Desktop\Lab2> python -m venv venv
PS C:\Users\student\Desktop\Lab2> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\student\Desktop\Lab2>
```

A red arrow points to the command `.\venv\Scripts\Activate.ps1` in the terminal.

Expected Output / Outcome

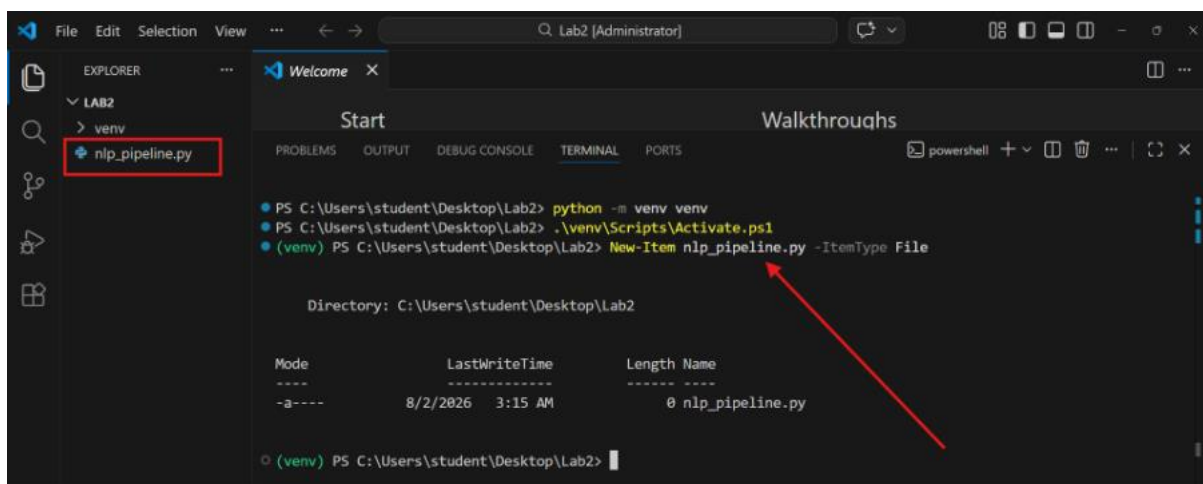
The virtual environment is activated and the terminal prompt displays (venv) before the current directory path, for example (venv) PS C:\Users\you\Desktop\Lab2>. This confirms any package installs or script runs from here stay isolated to this project.

Step 5 — Create the NLP Pipeline Script File

Execution

Run the following command in the VS Code terminal:

```
New-Item nlp_pipeline.py -ItemType File
```



```
File Edit Selection View ... Lab2 [Administrator]
EXPLORER LAB2
  venv
    nlp_pipeline.py
Welcome
Start Walkthroughs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\student\Desktop\Lab2> python -m venv venv
PS C:\Users\student\Desktop\Lab2> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\student\Desktop\Lab2> New-Item nlp_pipeline.py -ItemType File

Directory: C:\Users\student\Desktop\Lab2

Mode                LastWriteTime         Length Name
----                -
-a-----         8/2/2026   3:15 AM              0 nlp_pipeline.py

(venv) PS C:\Users\student\Desktop\Lab2>
```

A red arrow points to the command `New-Item nlp_pipeline.py -ItemType File` in the terminal. The Explorer pane on the left shows the file `nlp_pipeline.py` created inside the `venv` folder.

Expected Output / Outcome

A new file named `nlp_pipeline.py` is created inside the Lab2 folder and opens in the VS Code editor, or shows up in the Explorer pane ready to be opened. The file starts out empty (0 bytes), which is expected at this point.

Step 6 — Configure Script Headers and Raw Corpus

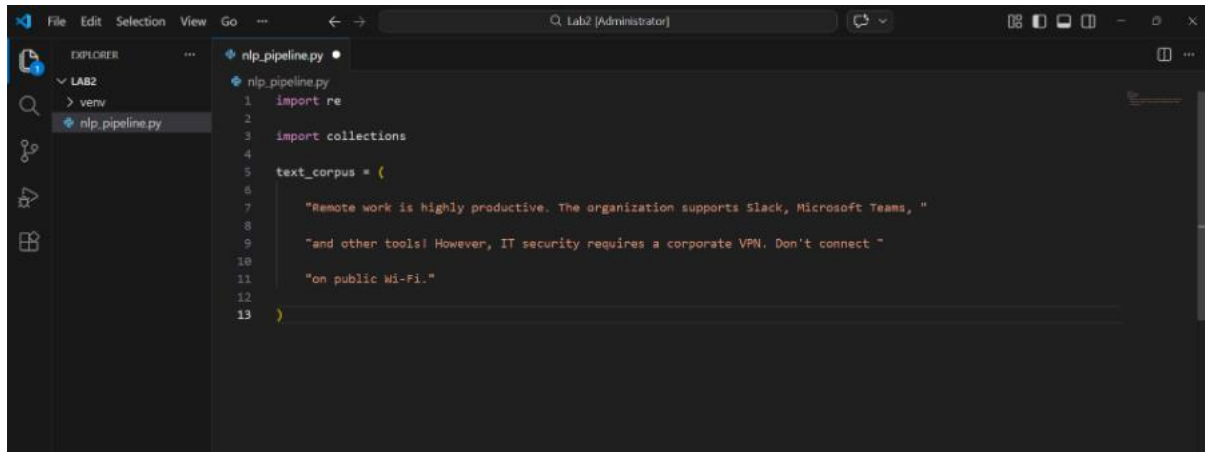
Execution

Open `nlp_pipeline.py` by double-clicking `nlp_pipeline.py` in the VS Code Explorer pane on the left side and add:

```
import re # lets us split and search text using patterns
import collections # gives us defaultdict, handy for counting things
later

text_corpus = ( # the raw block of text this whole script works on
```

```
"Remote work is highly productive. The organization supports Slack,
Microsoft Teams, "
    "and other tools! However, IT security requires a corporate VPN.
Don't connect "
    "on public Wi-Fi."
)
```



Expected Output / Outcome

The required import statements and sample text corpus are added to the script with no errors shown in the editor. Since nothing is printed yet at this stage, running the file now would simply do nothing visible, which is expected before Step 7 adds the first print statement.

Step 7 — Implement Sentence Segmentation

Execution

Add:

```
print("Part 1: Sentence Segmentation") # just a header so the output is
easy to read

sentences = re.split(r'(?<=[.!?])\s+', text_corpus) # split after . !
or ? followed by a space, keeping the punctuation attached

for idx, sentence in enumerate(sentences):
    print(f"Sentence {idx+1}: {sentence}")
```

Expected Output / Outcome

The text corpus is split into individual sentences based on sentence-ending punctuation. Running the script at this point prints exactly 4 sentences:

- Sentence 1: Remote work is highly productive.
- Sentence 2: The organization supports Slack, Microsoft Teams, and other tools!
- Sentence 3: However, IT security requires a corporate VPN.
- Sentence 4: Don't connect on public Wi-Fi.

Step 8 — Configure Word Tokenization

Execution

Add:

```
print("\nPart 2: Word Tokenization") # \n adds a blank line so this
section is visually separate

target_sentence = sentences[1] # pick sentence 2 (index 1) to run
tokenization tests on

print(f"Target: '{target_sentence}'")

naive_tokens = target_sentence.split() # the simplest possible
tokenizer: just split on whitespace

print(f"Naive Tokens ({len(naive_tokens)}): {naive_tokens}")
```

Expected Output / Outcome

The target sentence is selected and tokenized using whitespace only. With sentence 2 ("The organization supports Slack, Microsoft Teams, and other tools!"), this naive split produces **9** tokens, and commas or exclamation marks stay glued to the words next to them, for example 'Slack,' and 'tools!' come out as single tokens rather than being separated from the punctuation.

Step 9 — Implement Regex Word Tokenization

Execution

Add:

```
regex_tokens = re.findall(r"\w+(?:'\w+)?|[\.!?,\]", target_sentence) #
grabs words (keeping contractions like don't whole) and punctuation as
separate tokens

print(f"Regex Tokens ({len(regex_tokens)}): {regex_tokens}")
```

Expected Output / Outcome

Words and punctuation are extracted as separate tokens this time, instead of staying stuck together. The same target sentence now produces **12** tokens, since the commas after "Slack" and "Teams" and the exclamation mark after "tools" each become their own token.

Step 10 — Configure Stopword Removal and Normalization

Execution

Add:

```
print("\nPart 3: Normalization & Stopword Removal")

stopwords = {"is", "and", "other", "a", "on"} # small, common words we
don't care about for this exercise

normalized_tokens = [token.lower() for token in regex_tokens] #
lowercase everything so 'The' and 'the' count as the same word
```

```

cleaned_tokens = [
    token for token in normalized_tokens
    if token not in stopwords and token.isalnum() # drop stopwords and
drop anything that isn't purely letters/numbers (so punctuation gets
removed here)
]

print(f"Original Regex Tokens: {regex_tokens}")
print(f"Cleaned Tokens: {cleaned_tokens}")

```

Expected Output / Outcome

Tokens are normalized, stopwords are removed, and punctuation is filtered. The cleaned output for this sentence comes out to:

```
['the', 'organization', 'supports', 'slack', 'microsoft', 'teams', 'tools']
```

Notice "and" and "other" are gone (they're in the stopwords list), and the commas and exclamation mark from the regex step are gone too, since punctuation fails the `isalnum()` check.

Step 11 — Configure BPE Initial Vocabulary

Execution

Add:

```

print("\nPart 4: Byte Pair Encoding (BPE) Concept")

word_corpus = [ # each word broken into single characters, with </w>
marking the end of the word
    "l o o k </w>",
    "l o o k s </w>",
    "l o o k i n g </w>",
    "c o o k </w>"
]

print(f"Initial Vocabulary: {word_corpus}")

```

Expected Output / Outcome

The initial BPE vocabulary is configured and printed exactly as written above: four words, each spelled out as space-separated characters ending in the `</w>` marker. This is the starting point before any merges happen.

Step 12 — Implement BPE Pair Frequency Statistics

Execution

Add:

```

def get_pair_statistics(corpus): # counts how often every neighboring
pair of symbols shows up across all words
    pairs = collections.defaultdict(int) # stores pair frequencies

```

```

    for word in corpus:
        symbols = word.split() # break the word back into its
individual symbols

        for i in range(len(symbols) - 1): # walk through the word one
pair at a time
            pairs[(symbols[i], symbols[i + 1])] += 1 # tally this pair

    return pairs

```

Expected Output / Outcome

The function calculates adjacent character pair frequencies but produces no printed output on its own, since defining a function doesn't run it. It only does something once it's called, which happens in Step 14.

Step 13 — Implement BPE Merge Replacement Logic

Execution

Add:

```

def merge_character_pair(pair, corpus): # takes the winning pair and
glues it together everywhere it appears
    merged_corpus = []

    bigram = ' '.join(pair) # rebuild the pair as it currently looks,
e.g. 'o o'
    merged_str = ''.join(pair) # and how it should look after merging,
e.g. 'oo'

    pattern = re.compile(
        r'(?<!\S)' + re.escape(bigram) + r'(?!\S)' # create a pattern
to find the selected pair
    )

    for word in corpus:
        merged_word = pattern.sub(merged_str, word) # swap the pair for
its merged version in this word
        merged_corpus.append(merged_word)

    return merged_corpus

```

Expected Output / Outcome

The function merges adjacent character pairs in the vocabulary, but like the previous step, defining it doesn't execute anything yet. It runs for real once the loop in Step 14 calls it.

Step 14 — Implement BPE Iteration Loop

Execution

Add:


```

for iteration in range(3): # run 3 rounds of merging, one merge per
round
    pairs = get_pair_statistics(word_corpus) # see which pairs are most
common right now

    if not pairs: # safety check: stop early if there's nothing left to
merge
        break

    best_pair = max(pairs, key=pairs.get) # find the most common pair
    frequency = pairs[best_pair] # and remember how many times it
showed up

    word_corpus = merge_character_pair(best_pair, word_corpus) # apply
that merge across the whole vocabulary

    print(f"\nIteration {iteration + 1}: Merging pair {best_pair}
(Frequency: {frequency})")
    print(f"Updated Vocabulary: {word_corpus}")

```

Expected Output / Outcome

The BPE merge process executes and updates the vocabulary over 3 iterations. Running the script prints the following progression:

- Iteration 1: merges ('o', 'o'), frequency 4 → ['l oo k </w>', 'l oo k s </w>', 'l oo k i n g </w>', 'c oo k </w>']
- Iteration 2: merges ('oo', 'k'), frequency 4 → ['l ook </w>', 'l ook s </w>', 'l ook i n g </w>', 'c ook </w>']
- Iteration 3: merges ('l', 'ook'), frequency 3 → ['l look </w>', 'l look s </w>', 'l look i n g </w>', 'c ook </w>']

By the third merge, "look" has fully formed as a single unit in three of the four words, showing how BPE gradually builds up whole words from individual characters based on how frequently pairs occur together.

Step 15 — Run the NLP Pipeline Script

Execution

```
python nlp_pipeline.py
```

Expected Output / Outcome

The script executes successfully and displays, in order:

- Sentence segmentation results (4 sentences)
- Naive tokenization output (9 tokens for the target sentence)
- Regex tokenization output (12 tokens for the same sentence)
- Normalized and filtered tokens (7 cleaned tokens, stopwords and punctuation removed)
- BPE merge iterations and updated vocabulary (3 merge rounds, ending with "look" fully merged)

Validation Checkpoints

Verify Pipeline Output Logs

Execution

```
python nlp_pipeline.py
```

Expected Output / Outcome

The script runs successfully and displays all four sections in order, with no tracebacks or error messages:

- Sentence Segmentation
- Word Tokenization
- Normalization & Stopword Removal
- Byte Pair Encoding (BPE)

Verify Sentence Count

Execution

```
python -c "import re; corpus='Remote work is highly productive. The organization supports Slack, Microsoft Teams, and other tools! However, IT security requires a corporate VPN. Don''t connect on public Wi-Fi.'; print(len(re.split(r'(?<=[.!?])\s+', corpus)))"
```

Expected Output / Outcome

4 — confirming the corpus splits into exactly four sentences, matching what Step 7 printed inside the full script.

Verify BPE Subword Merge Output

Execution

```
python -c "import re, collections; from nlp_pipeline import merge_character_pair; corpus=['l o o k </w>']; p=('o','o'); print(merge_character_pair(p, corpus)[0])"
```

Expected Output / Outcome

l oo k </w> — confirming the merge function correctly glues a single ('o', 'o') pair together on an isolated test word, independent of the full iteration loop.

Cleanup Steps

Step 1 — Deactivate Virtual Environment

Execution

```
deactivate
```

Expected Output / Outcome

The (venv) prefix is removed from the terminal prompt, confirming the shell has returned to the system-wide Python environment.

Step 2 — Navigate Out of the Lab Folder

Execution

```
cd ..
```

Expected Output / Outcome

The terminal moves from the Lab2 folder to the parent directory, and the prompt path updates to reflect the new location.

Step 3 — Delete the Lab Folder

Execution

Delete the Lab2 folder using File Explorer.

Expected Output / Outcome

The Lab2 folder and its contents, including the venv environment and nlp_pipeline.py, are removed from the system, leaving the Desktop clean.

Final Lab Outcome

You successfully created a Python-based NLP pipeline, performed sentence segmentation, tokenization, normalization, stopword removal, and implemented a Byte Pair Encoding (BPE) workflow to demonstrate subword tokenization concepts.